



ENGIN 604 INTRODUCTION TO PYTHON FOR FINANCE

SEQUENCES & DATA STRUCTURES

Professor: *Gabriel E. Cabrera*
Teaching Assistant: *Alex Den Braber*



1 Basic Data Structures

As a general rule, data structures are objects that contain other objects (e.g. scalars) as sequences. Python provides the following *built-in* data structures:

Table 1: Data Structure Types

Structure	Example	Brackets	index	Repeated Elements	Mutable	Set Operations
List	A = [1,2,'hello']	[]	Yes	Yes	Yes	No
Tuple	B = (3,3,5)	()	Yes	Yes	No	No
Dictionary	D = {'Day':01,'Month':3}	{ }	Yes	Yes	Yes	No
Set	C = {6,7,8}	{ }	No	No	No	Yes

1.1 List (`list`)

- Build a list containing the names of the following technology companies: 'Facebook', 'Apple', 'Amazon' and 'Netflix'.
 - Print the first and last element of the list you created.
 - Print the first and last element of the list you created using negative indices.
 - Add "Nvidia" at the beginning of the list. How would you add it in the fourth position?
 - Add "Google" at the end of the list.
 - How many elements does the list have so far?
 - What position is the company 'Apple' at?
 - Remove "Nvidia" and "Netflix" from the list.
 - Sort the list in descending order according to the number of characters of each element in the list.
- Build the following lists:
 - part_a = [[0, 'a'], [1, 'b']]
 - part_b = [[2, 'c'], [3, 'd']]
 - Concatenate or combine both lists. What is the difference between concatenating with `+` and with `extend`?

- b. Print the second element of the list generated in (a).
3. Generate a list containing the numbers from 1 to 10.
 - a. Sort the list in descending order. What is the difference between sorting with `sort` and with `sorted`?
 - b. Select the values whose index is even.

1.2 Tuple (tuple)

A tuple (tuple) is created if you use a comma (,) to separate values when assigning to a single variable:

```
tuple_ = 1, 2, 'Facebook', 'Amazon'
tuple_
```

```
## (1, 2, 'Facebook', 'Amazon')
```

They can even store lists (list):

```
nested_tuple2 = (1,2), ('Facebook', 'Amazon'), ['Apple', 'Netflix']
nested_tuple2
```

```
## ((1, 2), ('Facebook', 'Amazon'), ['Apple', 'Netflix'])
```

You can only modify an element of the tuple (tuple) that allows modification, for example a list (list):

```
nested_tuple2[2].append('Google')
nested_tuple2
```

```
## ((1, 2), ('Facebook', 'Amazon'), ['Apple', 'Netflix', 'Google'])
```

It allows concatenation using the + sign:

```
('Facebook', 'Amazon') + ('Apple', 'Netflix')
```

```
## ('Facebook', 'Amazon', 'Apple', 'Netflix')
```

And even multiplication of elements using *:

```
('Facebook', 'Amazon') * 3
```

```
## ('Facebook', 'Amazon', 'Facebook', 'Amazon', 'Facebook', 'Amazon')
```

To force a list (list) to become a tuple (tuple) it is enough to write:

```
tuple([1, 2, 3, 4])
```

```
## (1, 2, 3, 4)
```

1.3 Dictionary (dict)

Dictionaries (dict) are mutable data structures, characterized by the use of key-value pairs (*key-value*).

```
dict1 = {'Name' : 'Janet Yellen',
        'Country' : 'United States',
        'Profession' : ' United States secretary of the treasury',
        'Age' : 74}
```

```
type(dict1)
```

```
## <class 'dict'>
```

To access the values associated with a key, it is enough to use [] and write the name of the key:

```
print(dict1['Name'], dict1['Age'])
```

```
## Janet Yellen 74
```

It inherits the `.keys()` method to access the names of the keys:

```
dict1.keys() # keys
```

```
## dict_keys(['Name', 'Country', 'Profession', 'Age'])
```

It inherits the `.values()` method to access the values associated with the keys:

```
dict1.values() # values
```

```
## dict_values(['Janet Yellen', 'United States', ' United States secretary of the treasury', 74])
```

With `.items()` you access both the key (*key*) and the values (*values*):

```
dict1.items() # items = keys + values
```

1.4 Set (set)

The set (`set`) data structure allows only unique values. To create a set (`set`), you create a list and then apply `set()`:

```
set1 = set([1, 2, 3, 4, 5, 5])  
set1
```

```
## {1, 2, 3, 4, 5}
```

```
set2 = set([3, 4, 5, 6, 9, 9, 7])  
set2
```

```
## {3, 4, 5, 6, 7, 9}
```

We can apply set operations:

- All the items that are in `set1` and `set2`:

```
# union  
set1.union(set2)
```

```
## {1, 2, 3, 4, 5, 6, 7, 9}
```

- Items that are in both `set1` and `set2`

```
# intersection  
set1.intersection(set2)
```

```
## {3, 4, 5}
```

- Items that are in `set1` but not in `set2`:

```
set1.difference(set2)
```

```
## {1, 2}
```

- Items that are in `set2` but not in `set1`:

```
set2.difference(set1)
```

```
## {9, 6, 7}
```

- Items that are in `set1` or `set2` but not in both:

```
set1.symmetric_difference(set2)
```

```
## {1, 2, 6, 7, 9}
```